Module 2: Privileged Containers & Attack Vectors — Complete Metadata

---

Module Overview

Field	Value
Module Number	2
Module Title	Privileged Containers & Attack Vectors
Total Duration	~35:00 (including module summary)
Number of Videos	4 lessons + 1 module summary
Difficulty Level	Intermediate–Advanced
Assessment	3 module MCQs + 2 final assessment MCQs

Module Summary Video

Field	Value
Title	Module 2 Overview: Privileged Containers & Attack Vectors
File	`module_summary_script.md`
Duration	2:30
Description	Introduces the module’s offensive security focus, previews attack demonstrations including host compromise, privilege escalation, container escape, and real-world CVE exploitation. Sets context with real-world breach examples and lists prerequisites.
Learning Objectives	Understand module scope and offensive focus; Identify real-world container security incidents; Review prerequisites
Prerequisites	Module 1 completion
Resources	Lab environment setup, ethical hacking guidelines
Screen Mix	PPT 80%, Terminal 20%
Resolution	1920x1080 @ 30fps

Video 2.1: What Privileged Containers Are and Why They’re Dangerous

Field	Value
Title	What Privileged Containers Are and Why They’re Dangerous
File	`video_01_script.md`
Duration	8:00
Description	Systematically examines what the `--privileged` flag changes: capability grant (14 → 41), Seccomp disablement, AppArmor removal, full device access, writable /proc and /sys, and kernel module loading. Compares default vs. privileged containers side by side with terminal evidence. Covers legitimate use cases and their safer alternatives. Includes a detection script for auditing privileged containers.
Learning Objectives	1. List all security mechanisms disabled by `--privileged`; 2. Compare default and privileged container capabilities; 3. Identify legitimate use cases and their alternatives; 4. Detect privileged containers using Docker inspect and custom scripts
Prerequisites	Module 1 (namespaces, cgroups, capabilities concepts)
Resources	Linux capabilities man page, Docker security documentation, CIS Docker Benchmark
Key Commands	`capsh --print`, `cat /proc/1/status`, `ls /dev/`, `docker inspect --format`, detection script
Screen Mix	PPT 25%, Terminal 40%, Code Editor 20%, Browser 15%
Resolution	1920x1080 @ 30fps

Video 2.2: Host System Compromise from Privileged Containers

Field	Value
Title	Host System Compromise Demonstrations
File	`video_02_script.md`
Duration	8:00
Description	Complete walkthrough of host compromise from inside a privileged container: disk enumeration and host filesystem mounting, credential harvesting (shadow file, SSH keys, cloud credentials), establishing three persistence mechanisms (SSH key, backdoor user, cron reverse shell), pivoting to other containers via Docker socket, and an alternative escape using the cgroup release_agent technique.
Learning Objectives	1. Mount and access the host filesystem from a privileged container; 2. Harvest credentials from the host; 3. Establish multiple persistence mechanisms; 4. Pivot from one container to access all containers on the host; 5. Execute the cgroup release_agent escape technique
Prerequisites	Video 2.1
Resources	MITRE ATT&CK for Containers (T1611), Felix Wilhelm’s cgroup escape research
Key Commands	`fdisk -l`, `mount /dev/sda`, `chroot`, `nsenter`, cgroup release_agent technique
Screen Mix	PPT 15%, Terminal 50%, Code Editor 25%, Browser 10%
Resolution	1920x1080 @ 30fps
Safety Note	All demonstrations must be in isolated lab environments; add persistent lab environment overlay

Video 2.3: Privilege Escalation and Container Escape Techniques

Field	Value
Title	Privilege Escalation and Container Escape Techniques
File	`video_03_script.md`
Duration	8:00
Description	Demonstrates four container escape techniques from non-privileged starting points: CAP_SYS_ADMIN cgroup escape, Docker socket mount exploitation, host PID namespace credential harvesting, and writable host path traversal. Provides a technique comparison matrix covering required configuration, complexity, and detection difficulty.
Learning Objectives	1. Execute container escape via CAP_SYS_ADMIN alone; 2. Escalate from Docker socket access to host root; 3. Harvest credentials via host PID namespace; 4. Exploit writable host path mounts for persistence; 5. Compare escape techniques by requirements and complexity
Prerequisites	Videos 2.1, 2.2
Resources	Trail of Bits container security research, Docker socket proxy documentation
Key Commands	`mount -t cgroup`, `docker run` (from socket), `cat /proc/[pid]/environ`, path traversal techniques
Screen Mix	PPT 20%, Terminal 45%, Code Editor 25%, Browser 10%
Resolution	1920x1080 @ 30fps

Video 2.4: Real-World CVE Analysis and Exploitation

Field	Value
Title	Real-World CVE Analysis and Exploitation
File	`video_04_script.md`
Duration	8:30
Description	In-depth analysis of three container escape CVEs across different stack layers: CVE-2019-5736 (runc binary overwrite — runtime layer), CVE-2020-15257 (containerd abstract socket access via host networking — manager layer), and CVE-2022-0185 (Linux kernel heap overflow — kernel layer). For each CVE, covers the vulnerability mechanism, exploit chain, patch, and detection strategy. Includes a CVE response playbook and comparison matrix.
Learning Objectives	1. Analyze CVE-2019-5736 exploit chain and the /proc/self/exe attack; 2. Understand CVE-2020-15257 namespace confusion with abstract Unix sockets; 3. Trace CVE-2022-0185 kernel heap exploitation steps; 4. Compare CVEs across attack layers, complexity, and mitigations; 5. Build a container CVE response playbook
Prerequisites	Videos 2.1–2.3, Module 1 (kernel and namespace concepts)
Resources	NVD entries for all three CVEs, runc security advisories, containerd security advisories, kernel.org advisories
Key Commands	`runc --version`, `containerd --version`, `uname -r`, `sysctl kernel.unprivileged_userns_clone`
Screen Mix	PPT 20%, Terminal 35%, Code Editor 25%, Browser 20%
Resolution	1920x1080 @ 30fps

Technical Production Requirements

Requirement	Specification
Resolution	1920x1080 Full HD
Frame Rate	30fps constant
Terminal Theme	Dark (Dracula or One Dark), 150% zoom
Code Editor	VS Code, 150% zoom, max 20–22 lines visible
PPT Slides	Maximized, 150% for readability, red accent theme for warnings
Browser	Chromium/Chrome, 150% zoom, pre-loaded pages
Screen Mix (Module Avg)	Terminal 42%, Code Editor 24%, PPT 20%, Browser 14%
Host OS	Ubuntu 22.04 LTS (isolated lab environment)
Docker Version	24.x+ with Docker Compose v2
Safety	Persistent “⚠️ LAB ENVIRONMENT” overlay on all demo segments
Pre-pulled Images	alpine, docker:latest, vulnerables/web-dvwa, aquasec/trivy, postgres
Ethical Disclaimer	Required at beginning and end of each video with terminal demonstrations